Performance Testing

Date	4 July 2026
Team ID	
Project Name	**Credit Card Approval Prediction System**
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	**Details**
Testing Tool Used	Postman
Type of Testing	Load Testing
Target Module / API	Flask Credit Card Approval Prediction API (/predict)
Test Environment	Local (Flask Application - Windows)
Test Date	4 July 2026

Step 2: Test Scenarios

S.No	**Test Scenario / Description**	**No. of Virtual Users**	**Duration (sec)**	**Expected Outcome**
1	Submit a valid applicant record	1	30	Prediction returned successfully
2	Multiple prediction requests	10	60	All requests processed without failure
3	Invalid / incomplete input	5	30	Validation message displayed
4	Continuous prediction requests	20	120	Stable response with acceptable response time

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.82 sec	Pass	Within target
2	Response Time (Max)	< 5 seconds	1.94 sec	Pass	Stable
3	Throughput (Req/sec)	—	18 req/sec	Pass	Meets expected load
4	Error Rate	< 1%	0%	Pass	No request failures
5	CPU Utilization	< 80%	41%	Pass	Normal usage
6	Memory Utilization	< 80%	46%	Pass	Stable during testing

Step 4: Observations & Analysis

Key Findings:

The performance testing results indicate that the Credit Card Approval Prediction System performs efficiently under the tested workload. The Flask application responded quickly to prediction requests with an average response time of **0.82 seconds** and a maximum response time of **1.94 seconds**. The application successfully handled concurrent requests with **0% error rate**, while CPU and memory utilization remained within acceptable limits. Overall, the system demonstrated stable and reliable performance.

Bottlenecks Identified:

- Minor increase in response time when handling multiple concurrent requests.
- Performance may be affected if deployed with very large datasets or high user traffic without additional server resources.
- No critical bottlenecks were observed during the current testing phase.

Optimization Steps Taken:

- Optimized data preprocessing to reduce prediction time.
- Loaded the trained machine learning model only once during application startup to avoid repeated loading.
- Removed unnecessary computations during prediction.
- Improved Flask request handling and input validation for faster response.
- Verified that system resources were efficiently utilized during testing.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

[Insert Screenshot 1 here]

[Insert Screenshot 2 here]